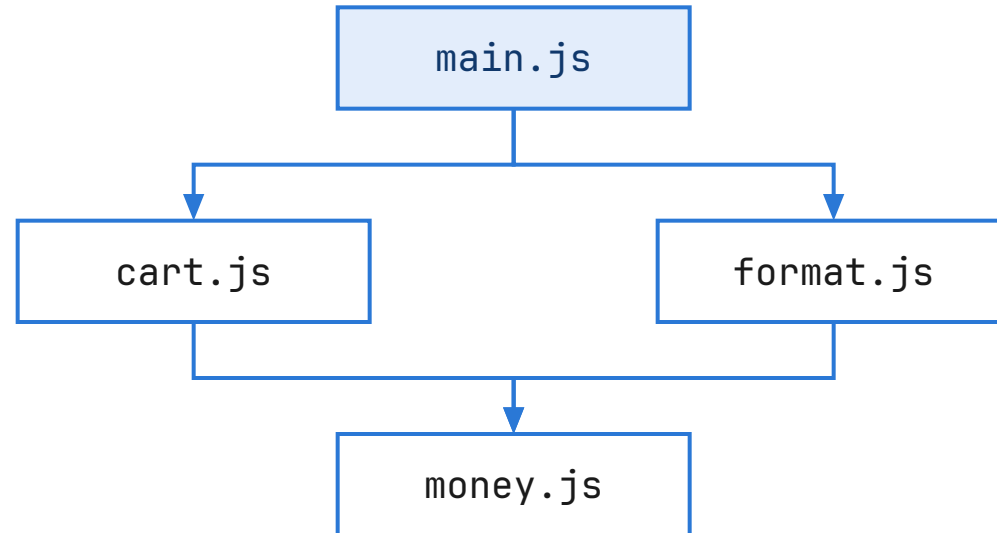


The Module Graph

Static imports are resolved before any of your code runs. Dynamic `import()` happens later, and returns a promise.

Static: `import ... from '...'`



The whole graph is discovered and fetched first. Imports are hoisted, so their position in the file does not matter. Each module is evaluated exactly ONCE, however many files import it.

Dynamic: `await import('...')`

App starts: heavy module NOT loaded

User clicks something

```
const mod = await import('./chart.js')
```

Fetches now, then used

A function call, not a declaration, so it can sit inside an if, take a computed path, and be awaited. This is code splitting.

ESM (`import / export`) is the standard, and the default everywhere. CommonJS (`require / module.exports`) is Node's older system: `require` is synchronous and runs at the point it appears, which is exactly why `import` can be analysed ahead of time and `require` cannot. In Node, `.mjs` or `"type": "module"` selects ESM; in the browser, `<script type="module">`.